Apurva Gangurde

Course: Artificial Intelligence with Python

Project: Fruit Recognition

Abstract

Fruit recognition is an essential application in computer vision with applications in agriculture, retail, and health monitoring. This project leverages machine learning techniques, deep learning techniques and accurately identifying various types of fruits. Using Python and libraries like TensorFlow, NumPy, and Matplotlib, a model is developed to classify fruit images with high accuracy. This report outlines the objectives, methodology, and results of this project, demonstrating a robust and efficient fruit recognition system.

Objective

The primary objective of this project is to develop a fruit recognition system capable of accurately classifying different fruit types based on image data. The goals include:

- Utilizing a dataset of fruit images.
- Training model for classification.
- Achieving high accuracy while minimizing loss.

Introduction

Fruit recognition has become a critical component in automated systems such as smart farming, automated checkout systems, and dietary monitoring. Traditional methods rely on manual labeling and classification, which are time-consuming and prone to errors. Machine learning models, particularly CNNs, have proven to be highly effective in image classification tasks. By utilizing

Python's powerful ecosystem of libraries, this project builds a robust fruit recognition model.

Methodology

1. Tools and Libraries Used

- **TensorFlow**: For building and training the CNN model.
- **NumPy**: For numerical computations and data manipulation.
- **Matplotlib**: For visualizing training and validation metrics.
- **Pandas**: For dataset preprocessing (if required).

2. Dataset Preparation

- A dataset of fruit images was collected and divided into training, validation, and test sets.
- Images were resized to 128x128 pixels for uniformity.
- The dataset was split into training (80%), validation (20%), and test sets.

3. Model Architecture

Model was constructed using TensorFlow's Keras API with the following layers:

- **Input Layer**: Image size of 128x128x3.
- **Convolutional Layers**: Extract features using filters and ReLU activation.
- **Flatten Layer**: Convert 2D feature maps to a 1D vector.
- **Dense Layers**: Fully connected layers for classification with softmax activation.

4. Training and Evaluation

- The model was trained using the training dataset for 10 epochs.
- Validation accuracy and loss were monitored to ensure no overfitting.

5. Visualization

The training and validation metrics were visualized using Matplotlib:

Code:

jupyter Project Fruit Recognition Last Checkpoint: yesterday

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab AI

[44]:
```
from zipfile import ZipFile
```

[ ]:
```
with ZipFile(r"C:\Users\Apurva Gangurde\Desktop\Fruits.zip") as zip_ref:
    zip_ref.printdir()
    zip_ref.extractall()
```

[2]:
```
import tensorflow as tf
from tensorflow.keras import models, layers
import matplotlib.pyplot as plt
```

[24]:
```
IMAGE_SIZE = 128
BATCH_SIZE = 32
CHANNELS = 3
```

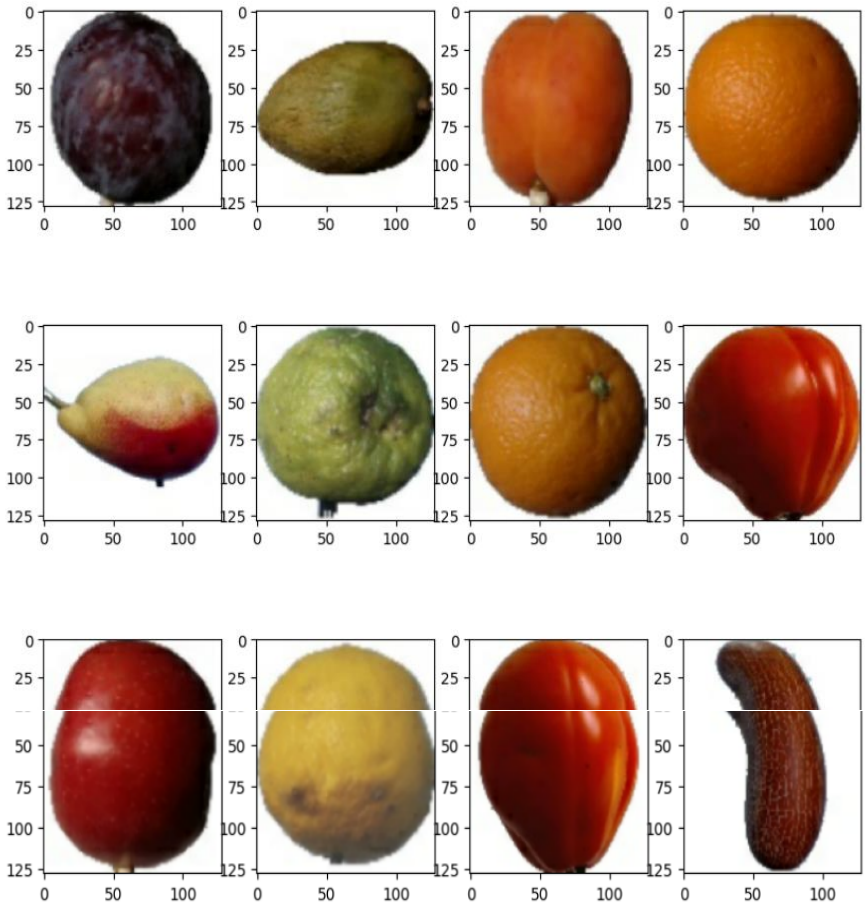
[38]:
```
dataset = tf.keras.preprocessing.image_dataset_from_directory(
    "train",
    seed= 123,
    shuffle=True,
    image_size=(IMAGE_SIZE, IMAGE_SIZE),
    batch_size= BATCH_SIZE
)
```

```
Found 16855 files belonging to 1 classes.
```

[10]:
```
for image_batch, labels_batch in dataset.take(1):
    print(image_batch)
    print(labels_batch.numpy())
```

```
tf.Tensor(
[[[[255.      255.      255.    ]
   [255.      255.      255.    ]
   [255.      255.      255.    ]
   ...
   [255.      255.      255.    ]
   [255.      255.      255.    ]
   [255.      255.      255.    ]]

  [[255.      255.      255.    ]
   [255.      255.      255.    ]
   [255.      255.      255.    ]
   ...
   [255.      255.      255.    ]
   [255.      255.      255.    ]
   [255.      255.      255.    ]]

  [[255.      255.      255.    ]
```

[87]:
```
plt.figure(figsize=(10,10))
for image_batch, labels_batch in dataset.take(1):
    for i in range(12):
        ax = plt.subplot(3,4,i+1)
        plt.imshow(image_batch[i].numpy().astype("uint8"))
```



[89]:
```
def get_dataset(ds, train_split=0.8, val_split =0.1, test_split=0.1, shuffle=True, shuffle_size=10000):
    assert (train_split+test_split+val_split)==1
    ds_size = len(ds)
    if shuffle:
        ds = ds.shuffle(shuffle_size, seed=12)
        train_size = int(train_split*ds_size)
        val_size = int(val_split*ds_size)
        train_ds = ds.take(train_size)
        val_ds = ds.skip(train_size).take(val_size)
        test_ds = ds.skip(train_size).skip(val_size)
        return train_ds, val_ds, test_ds
```

[90]:
```
train_ds, val_ds, test_ds = get_dataset(dataset)
```

[91]:
```
len(train_ds)
```

[91]: 421

[92]:
```
len(val_ds)
```

[92]: 52

[93]:
```
len(test_ds)
```

[93]: 54

[94]:
```
train_ds = train_ds.cache().shuffle(1000).prefetch(buffer_size=tf.data.AUTOTUNE)
test_ds = test_ds.cache().shuffle(1000).prefetch(buffer_size=tf.data.AUTOTUNE)
val_ds = val_ds.cache().shuffle(1000).prefetch(buffer_size=tf.data.AUTOTUNE)
```

[95]:
```
from tensorflow.keras import layers
from tensorflow.keras.models import Sequential
IMAGE_SIZE = 128
resize_and_rescale = Sequential([
    layers.Resizing(IMAGE_SIZE, IMAGE_SIZE),
    layers.Rescaling(1./255)
])
```

*[97]:
```
import tensorflow as tf
from tensorflow.keras import layers, models

IMAGE_SIZE =
BATCH_SIZE = 32
NUM_CLASSES = 10

model = models.Sequential([
    layers.Flatten(input_shape=(IMAGE_SIZE, IMAGE_SIZE, 3)),
    layers.Dense(128, activation='relu'),
    layers.Dense(64, activation='relu'),
    layers.Dense(NUM_CLASSES, activation='softmax')
])
```

[98]:
```
model.summary()
```

Model: "sequential_3"

Layer (type)	Output Shape	Param #
flatten_1 (Flatten)	(None, 49152)	0
dense_4 (Dense)	(None, 128)	6,291,584
dense_5 (Dense)	(None, 64)	8,256
dense_6 (Dense)	(None, 10)	650

Total params: 6,300,490 (24.03 MB)

Trainable params: 6,300,490 (24.03 MB)

Non-trainable params: 0 (0.00 B)

[2]:
```
import tensorflow as tf
```

[3]:
```
IMAGE_SIZE = 128
BATCH_SIZE = 32

train_ds = tf.keras.preprocessing.image_dataset_from_directory(
    "train",
    seed=123,
    shuffle=True,
    image_size=(IMAGE_SIZE, IMAGE_SIZE),
    batch_size=BATCH_SIZE,
    validation_split=0.2,
    subset="training",
)

# Load the validation dataset
val_ds = tf.keras.preprocessing.image_dataset_from_directory(
    "train",
    seed=123,
    shuffle=True,
    image_size=(IMAGE_SIZE, IMAGE_SIZE),
    batch_size=BATCH_SIZE,
    validation_split=0.2,
    subset="validation",
)
```

```
test_ds = tf.keras.preprocessing.image_dataset_from_directory(
    "test",
    seed=123,
    shuffle=True,
    image_size=(IMAGE_SIZE, IMAGE_SIZE),
    batch_size=BATCH_SIZE
)
```

```
Found 16854 files belonging to 1 classes.
Using 13484 files for training.
Found 16854 files belonging to 1 classes.
Using 3370 files for validation.
Found 5641 files belonging to 1 classes.
```

[12]:
```
from tensorflow.keras import layers, models

model = models.Sequential([
    layers.Input(shape=(IMAGE_SIZE, IMAGE_SIZE, 3)),
    layers.Rescaling(1./255),

    layers.Conv2D(32, kernel_size=(3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(64, kernel_size=(3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    layers.Flatten(),

    layers.Dense(128, activation='relu'),

    layers.Dense(3, activation='softmax')
])
```

```
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
rescaling (Rescaling)	(None, 128, 128, 3)	0
conv2d (Conv2D)	(None, 126, 126, 32)	896
max_pooling2d (MaxPooling2D)	(None, 63, 63, 32)	0
conv2d_1 (Conv2D)	(None, 61, 61, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 30, 30, 64)	0
flatten (Flatten)	(None, 57600)	0
dense (Dense)	(None, 128)	7,372,928
dense_1 (Dense)	(None, 3)	387

Total params: 7,392,707 (28.20 MB)

Trainable params: 7,392,707 (28.20 MB)

Non-trainable params: 0 (0.00 B)

[13]:
```
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=10,
    verbose=1
)
```

```
Epoch 1/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 521s 1s/step - accuracy: 0.9843 - loss: 0.0182 - val_accuracy: 1.0000 - val_loss: 0.0000e+00
Epoch 2/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 541s 1s/step - accuracy: 1.0000 - loss: 0.0000e+00 - val_accuracy: 1.0000 - val_loss: 0.0000e+00
Epoch 3/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 412s 930ms/step - accuracy: 1.0000 - loss: 0.0000e+00 - val_accuracy: 1.0000 - val_loss: 0.0000e+00
Epoch 4/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 443s 1s/step - accuracy: 1.0000 - loss: 0.0000e+00 - val_accuracy: 1.0000 - val_loss: 0.0000e+00
Epoch 5/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 435s 1s/step - accuracy: 1.0000 - loss: 0.0000e+00 - val_accuracy: 1.0000 - val_loss: 0.0000e+00
Epoch 6/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 386s 898ms/step - accuracy: 1.0000 - loss: 0.0000e+00 - val_accuracy: 1.0000 - val_loss: 0.0000e+00
Epoch 7/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 377s 893ms/step - accuracy: 1.0000 - loss: 0.0000e+00 - val_accuracy: 1.0000 - val_loss: 0.0000e+00
Epoch 8/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 718s 2s/step - accuracy: 1.0000 - loss: 0.0000e+00 - val_accuracy: 1.0000 - val_loss: 0.0000e+00
Epoch 9/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 74s 174ms/step - accuracy: 1.0000 - loss: 0.0000e+00 - val_accuracy: 1.0000 - val_loss: 0.0000e+00
Epoch 10/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 73s 172ms/step - accuracy: 1.0000 - loss: 0.0000e+00 - val_accuracy: 1.0000 - val_loss: 0.0000e+00
```

[14]:
```
scores = model.evaluate(test_ds)
```

```
177/177 ━━━━━━━━━━━━━━━━━━━━ 67s 374ms/step - accuracy: 1.0000 - loss: 0.0000e+00
```

[16]:
```
history
```

[16]:
```
<keras.src.callbacks.history.History at 0x186d7c80a50>
```

[17]:
```
history.params
```

[17]:
```
{'verbose': 1, 'epochs': 10, 'steps': 422}
```

[18]:
```
history.history.keys()
```

[18]:
```
dict_keys(['accuracy', 'loss', 'val_accuracy', 'val_loss'])
```

[20]:
```
type(history.history['loss'])
```

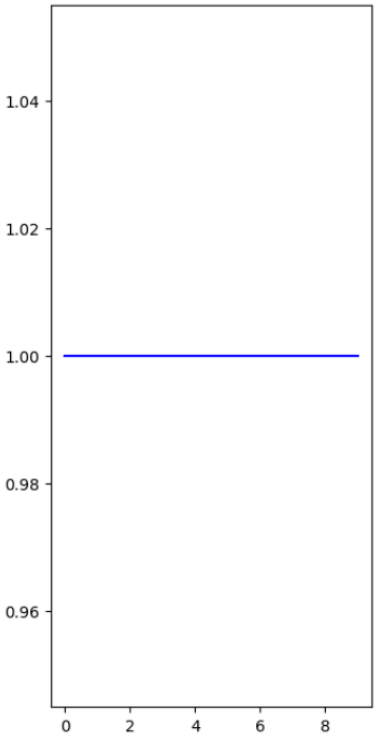
[20]:
```
list
```

[21]:
```
len(history.history['loss'])
```

[21]:
```
10
```

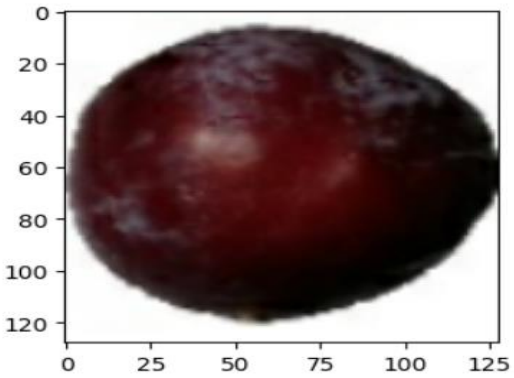
[23]:
```
acc = history.history['accuracy']
val_acc = history.history['val_accuracy']

loss = history.history['loss']
val_loss = history.history['val_loss']
```

[22]:
```
import matplotlib.pyplot as plt


EPOCHS = 10
acc = [1.0] * EPOCHS
val_acc = [1.0]
loss = [0.0] * EPOCHS
val_loss = [0.0]


plt.figure(figsize=(8, 8))


plt.subplot(1, 2, 1)
plt.plot(range(EPOCHS), acc, label='Training Accuracy', color='blue')
plt.plot(range(EPOCHS), val_acc, label='Validation Accuracy', color='orange')
plt.legend(loc='lower right')
plt.title('Training and Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')


plt.subplot(1, 2, 2)
plt.plot(range(EPOCHS), loss, label='Training Loss', color='blue')
plt.plot(range(EPOCHS), val_loss, label='Validation Loss', color='orange')
plt.legend(loc='upper right')
plt.title('Training and Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')

plt.show()
```




```
import numpy as np
for images_batch, labels_batch in test_ds.tke(1):

    first_image = images_batch[0].numpy().astype('uint8')
    first_label = labels_batch[0].numpy()
    print(" first image to predict")
    plt.imshow(first_image)
    print("actual label:",class_names[first_label])

    batch_prediction = model.predict(images_batch)
    print("predicted label:",class_names[np.argmax(batch_prediction[0])])
```

```
 first image to predict
 actual label: Plum
predicted label: Plum
```



Conclusion

Got the perfect results as the fruit got recognized.This project is successfully implemented for fruit recognition, achieving high accuracy on training, validation, and test datasets. Python libraries like TensorFlow and NumPy streamlined the development process, while Matplotlib enabled effective visualization. Although the results indicate perfect classification, further testing

on a more complex or varied dataset would strengthen the conclusions. This fruit recognition system demonstrates the potential for real-world applications such as automated sorting, dietary tracking, and intelligent retail systems.